1 Features of a TS

Univariate: A TS that consists of a single variable recorded sequentially over time (e.g., bike sale over time from a single store) Multivariate: Consists of multiple variables recorded sequentially over time (e.g., bike sale and profit over time from a single store) Hierachical: Consists of multiple variables from multiple observations, nested within the larger group categories, recorded sequentially over time (e.g., bike sale and profit from multiple stores over time)

1.1 Different tasks in TS analysis

Prediction/forecasting (supervised) Example: Predicting the number of tourists in the upcoming quarters, Approach: Statistical models (exponential smoothing, ARIMA, etc..) or ML/DL models (CNN,RNN,LightGBM, etc. . .)

Clustering/Anomaly detection (unsupervised) Example 1: Detect earthquakes, Approach: Start with some dimensionality reduction/transformation, compute similarity measures (euclidean, DTW), and apply unsupervised learning models (k-means, DBSCAN, etc.)

Factors that affect forecasting

How much (historical) data we have
How far into the future do we forecast
How similar historical patterns are compared to the future
How 'stable' the environment is
How much domain knowledge/information you possess

1.2 Working with TS - Visualizing

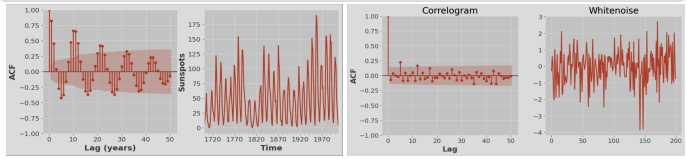
The first step in any TS analysis should be a time plot! **Temporal dependence** What makes TS different to data we've seen previously is that they include temporal dependence - observations close in time tend to be correlated. That means that the value of the series at time t often depends in some way on past values. We can quantify this dependence by looking at the correlation of a TS with "lagged" values of itself. We call this **autocorrelation**.

Correlogram Simpler plot to visualise all this information. Plots the autocorrelation function (ACF) on the y-axis and lags on the x-axis. The formula used for standard error depends upon the situation.

If ACF for residuals diagnostics as part of the ARIMA routine, the standard errors are determined assuming the residuals are white noise..

If ACF used as part of the raw data as a model selection tool, then we should use the Bartlett's formula. 'bartlett_confint=True'

```
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(sun["sunspots"], lags=50, title="ACF of Sunspots")
sun["sunspots"].plot.line(xlabel="Time", ylabel="Sunspots")
```



1.3 Time Series (TS) Patterns

Trend: long term increases or decreases in the series. Patterns that don't repeat with a regular period.

Seasonality: regular variation in the series at some fixed interval, e.g., month, day of week, time of day, etc.

Cyclicity: variations in the series that repeat with some regularity but of unknown and changing period.

Seasonal smaller timeframe than cyclical, but at a fixed frequency. Cyclical larger timeframe than seasons, and not at a fixed frequency. Cyclical always larger timeframe than seasonal behaviour.

White noise: Zero mean, constant variance, no autocorrelation, iid and Gaussian. If TS has white noise then that means it is a series of random numbers and cannot be predicted. Residuals/errors from a TS for a forecast model should resemble white noise, implies that we have extracted all the useful information from the time-series.

2 Time series decomposition

Trend-cycle (T) Seasonal (S) Remainder (R)

Additive $y_t = S_t + T_t + R_t$: Appropriate if the magnitude of the seasonal fluctuations, or the variation around the trend-cycle, does not vary with the value of the series.

Multiplicative $y_t = S_t \times T_t \times R_t$: Appropriate if variation in the seasonal pattern, or the variation around the trend-cycle, appears to be proportional to the value of series.

2.1 Estimating the trend-cycle

Curve-fitting Easily calculate and remove a simple polynomial curve from our data. Method assumes the same curve fits the data at all time points, but this is usually not the case

```
from statsmodels.tsa.tsatools import detrend
for order in range(3):
  fit_label = f"Poly. fit (order {order})"
  trend = detrend(df["CO2"], order=order)
  df[fit_label] = df["CO2"] - trend
  detrended_series = df["CO2"] - df[fit_label]
  df[["CO2", fit_label]].plot.line(title=f"Order {order}:
       Original", xlabel="Time", ylabel="CO2 (ppm)")
  detrended_series.plot.line(title=f"Order {order}: Detrended",
       xlabel="Time")
```

Moving average Estimates trend using a moving average which just means, passing a window of some size over the data and calculating the average . Nature of data, expert knowledge, tells window size. If seasonality in the data, use a window equal to the seasonal period. This will average out seasonal variation and allow us to focus on the trend-cycle. Period other than 12 may contaminate the trend-cycle with seasonality. **Problem with MA of even window size** moving average doesn't line up with the index of the original series. To rectify this, we usually take an extra 2-MA of the result of a MA with an even window.

2.2 Estimating the Seasonality

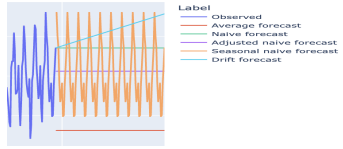
1. Remove the trend-cycle component from the data
2. Estimate the seasonal component by averaging

2.3 Estimating the remainder

To estimate the remainder, we simply subtract the trend-cycle and seasonal components from the original series:

2.4 Just use seasonal_decompose or STL

```
model = seasonal_decompose(co2[["CO2"]],model="additive",period=12)
model = STL(co2["CO2"], period=12, seasonal=13).fit() #STL
with mpl.rc_context():
    mpl.rc("figure", figsize=(7, 10))
    model.plot()
    plt.tight_layout()
```



3 Forecasting

Average Use the average of the series for all future forecasts.

Naive Use the last observation for all forecasts.

Seasonally-adjusted Naive seasonally adjusted by applying a classical decomposition.

Seasonal Naive Set each forecast as the last observed value from the same season of the year.

Drift Forecasts equal to last value in the series plus the average change of the series.

4 Exponential Model